

The five questions from the meeting

Short answers to say in the room, and the reasoning behind each one.

SOURCED means the research has a citation for it — you can defend it. **ASSUMPTION** means it is our own construction with no external source. Say so if challenged; the dossier already marks these, and conceding one honestly costs far less than being caught defending it as fact.

Two things changed as a result of this meeting.

Question 1's draft answer is off-message and needs replacing. "COBOL developers are dead, the codebase is hard to read" is not our problem statement, and if we say it in the room the pitch collapses in one follow-up question. Full reasoning below.

Questions 4 and 5 have no research behind them. The dossier deliberately declined to claim a revenue model, an ROI multiple or a payback period, because nothing sourced supports one. The answers below are new constructions built for this meeting. They are defensible as reasoning — they are not findings.

QUESTION 1

What problem are we actually solving?

Asked in the meeting as: only COBOL → Java conversion, or something else? Draft answer noted: "COBOL developers dead; codebase hard to read."

SAY THIS

We are not solving COBOL-to-Java conversion. That is already solved and commoditised — IBM, AWS and TCS all ship it. We solve the step after it: proving the new program behaves identically to the old one, when no specification was ever written to check it against. And we add one thing nobody else does — we also check the legacy program against what the regulator actually requires.

Why the draft answer has to go

Both halves of it are weak, and the second one is dangerous:

- **"COBOL developers are dead" is a contested claim and the wrong one.** The dossier (01 §2) marks every workforce statistic as *[contested]* — the retirement ages and headcounts circulate through vendor blogs and recruitment marketing, and the primary sources are old. More importantly, a bank *can* hire a COBOL contractor. The sharper and true version is: **the people who wrote the undocumented business rules are gone.** Nobody can hire back the engineer who decided in 1994 how to handle an account opened on 31 January.
- **"Hard to read" invites exactly the wrong solution.** If the problem is readability, the answer is a documentation generator — and that is Option A in Solution Folder/01 , which we *rejected*. The best published COBOL rule extractor reaches ~62% precision. A document that is a quarter wrong, with no marking of which quarter, is a liability generator, because an engineer builds on it.
- **Saying "we convert COBOL to Java" ends the pitch.** Any judge who has worked on mainframe modernisation — and Cognizant is the organiser — answers with "IBM watsonx Code Assistant already does that." We have no good reply, because they are right.

The problem in three layers, if you have more time

1. **The code is the specification, and there is no other one.** The business rules exist only inside programs written decades ago. Nothing states what they are supposed to do. **SOURCED**
2. **So translation is not the bottleneck — verification is.** Translation ships from three vendors. What nobody can do is prove the rewrite behaves identically, which is why generated Java does not reach production. IBM Research says of IBM's own translation product that the output cannot be trusted to correctly translate the original code. **SOURCED**
3. **And everyone assumes the legacy is correct by definition.** A program written in 1987 can breach a Master Direction issued decades later, and a faithful migration carries the breach forward. That is our addition. **SOURCED**

One-line version to memorise: we make behavioural equivalence something a bank can prove, then check that answer against the regulator.

QUESTION 2

Why did we choose this problem statement?

Draft answer noted: Indian systems outdated and bureaucratic; a gap in the Indian ecosystem; we have worked on a similar system before.

SAY THIS

Three reasons. It is a real and specifically Indian problem with money already committed to it — BCG puts the Indian core-banking upgrade bill at about a billion dollars. It fits the theme and the room, because mainframe modernisation is a service line the organiser sells. And it is one of the very few problems at this scale that a four-person student team can genuinely build against, because the entire toolchain is free and it runs on a laptop.

The four reasons, ranked by how well they hold up

Reason	Detail	Standing
It is real, and it is Indian	BCG: Indian lenders must invest ~\$1bn over 5–10 years on legacy cores. RBI froze HDFC Bank’s Digital 2.0 launches and card issuance from Dec 2020 into 2022. RBI’s IT Governance Master Direction (Nov 2023) requires a board-approved change-management policy. An IT failure here is a supervisory event.	SOURCED
It fits the theme and the panel	Banking and Finance Technology, with Cognizant as organiser — and Cognizant markets accelerators that extract business rules from legacy COBOL and rewrite the application. We are speaking to people who know this domain, which is why the honest positioning matters more than a novelty claim.	SOURCED
It is actually buildable by us	This is the reason that separates us from teams with a bigger idea and no build. GnuCOBOL, ProLeap, gcov, the NIST suite and AWS’s CardDemo corpus are all free and open. No mainframe, no vendor contract, no production data. Twelve weeks, four engineers, commodity laptops.	SOURCED ASSUMPTION on the sizing
We have adjacent experience	Use this only if you can name the specific system and what transferred from it. A vague “we worked on something similar” reads as filler; a concrete one-liner reads as credibility.	ASSUMPTION

Drop from the answer: “bureaucratic.” It sounds like a complaint about institutions rather than an engineering diagnosis, and a banking panel will hear it as naive. The precise version is that the bank’s refusal to touch the system is the *rational* decision under its own risk framework — which is a much stronger thing to say.

QUESTION 3

Why Java? Why not Python, C or C++?

Draft answer noted: because decimal arithmetic is very similar for both Java and COBOL.

SAY THIS

Two reasons, and then a caveat that matters more than both. Java is where the market actually migrates — IBM, AWS and TCS all target it, so matching that is what makes the tool relevant to a real programme. And IBM's own interoperability documentation names `BigDecimal` as the correct partner for COBOL's packed and zoned decimal, so there is a documented, exact mapping for money. But the important part is that our harness does not care: it compares the outputs of two executables, not the code. Java is the demo target because it is the market default, not because the method depends on it.

Sharpening the draft answer

“Decimal arithmetic is very similar” is directionally right but imprecise in a way that is easy to puncture. The precise version:

- **COBOL money fields are packed decimal** (`COMP-3`) — exact decimal representation, no binary-fraction error. `PIC 59(7)V99 COMP-3` means exactly seven integer digits and two decimals, and the compiler truncates at that boundary on every intermediate store. **SOURCED**
- **Java's `BigDecimal` is the documented legal partner** for zoned and packed decimal in IBM's COBOL/Java interop docs, and it is what Micro Focus COBOL uses under the covers when compiling to JVM bytecode. **SOURCED**
- **Do not say Java is unique in this.** Python's `decimal` module implements the same IEEE decimal standard and would work fine. If you claim only Java can do exact decimal, someone will correct you. The honest claim is that Java is the *market default* with a *documented mapping*, not that it is the only capable target.
- **C and C++ are the genuinely weak targets** — no exact decimal type in the standard library, so you would be hand-rolling fixed-point arithmetic, which is precisely the class of code the whole project exists to distrust.

The answer that turns this into a strength

The differential harness runs the legacy COBOL and the rewrite on the same inputs and compares outputs. It never reads the Java. That means the target language is a swappable choice, and if a bank migrates to something else, the method is unchanged. Say this out loud — it converts “why Java?” from a question about a constraint into a demonstration that the design is not coupled to one.

Bonus, and it is the demo: the divergence we show is *caused* by the decimal mismatch — the Java uses `RoundingMode.HALF_UP` while the COBOL truncates at the PIC boundary, giving ₹116 against ₹115. So the answer to “why Java?” leads directly into the most concrete thing we have.

QUESTION 4

How do we generate recurring revenue if this is a one-time job?

Raised as: this is sort of a one-time job.

Read this before using the answer below. The dossier records no revenue model, no pricing research and no costing work — section 3 of 08 - Business Plan Research.md says explicitly that no rupee totals appear on the deck because no costing research exists. Everything in this section is a construction built for this meeting. It is reasoning we can defend as reasoning; it is not a researched finding, and it must not be presented as one.

SAY THIS
The premise is right for a migration, and wrong for what we actually sell. The migration is the event; the regression suite and the compliance harness are the product, and both live for as long as the program does. Every future change to that program re-runs the suite, and every time the RBI amends a Direction the whole portfolio has to be re-checked against the new rule. We are not selling a one-time translation. We are selling the thing a bank has to keep doing.

Four recurring lines, strongest first

- 1. **Change assurance — the suite outlives the migration.** This is the real answer. Once a program has a spec and a test corpus, every subsequent change to it needs re-verification, and banks change these programs continuously. The tool stops being a migration instrument and becomes standing regression infrastructure in the bank’s CI. That is a subscription, not a project.
- 2. **Regulatory re-verification.** The third reference is an executable model of an RBI Master Direction. Regulators amend Directions. Every amendment invalidates the previous compliance verdict across every program that touches the affected rule, and the portfolio has to be re-run. This is recurring because the regulator, not the bank, triggers it — which makes it a much better revenue line than one the customer can defer.
- 3. **It is per program family, not per bank.** A bank does not have one COBOL program; it has hundreds. Verifying one accrual family is one engagement. The estate is the portfolio, and it is consumed over years, not in one cutover.
- 4. **Migrations themselves are multi-year.** Assurance is consumed continuously through the programme, not once at the end — every batch of translated programs needs its own verdict.

Plausible commercial shapes **ALL ASSUMPTION**

Shape	How it works	Recurring?
Annual licence per program family under management	The bank pays to keep a family under continuous verification; the harness runs on every change	Yes — the core line
Initial engagement fee	One-off build of the spec and input corpus for a new program family	No — but it recurs per family
Regulatory update service	We maintain the executable Direction models as RBI amends them, and re-run affected portfolios	Yes
Licensed to the SI, not the bank	Cognizant, TCS or Infosys embed the harness in their own migration delivery and pay per programme	Yes — and it is the fastest route to scale

If a judge presses for numbers: say plainly that we have not done pricing research and will not invent a figure, then give the reasoning above. “We have not priced it yet” from a student team is entirely credible. A fabricated revenue-per-customer figure that falls apart under one follow-up question is not.
The line that reframes the objection: the migration is the occasion we arrive on; the obligation to keep proving the system is correct is what we sell.

QUESTION 5

Why would a company buy this? What do they get, financially?

Raised as: how much would migration cost, versus how much the company benefits from our product.

The honest position. The research explicitly declines to claim any ROI multiple, payback period or revenue figure — nothing sourced supports one — and it has no per-bank migration cost either. So we cannot answer the cost-versus-benefit comparison as a ratio. What we *can* do is price the downside, which is well documented, and name the benefits with mechanisms rather than multiples. Attempting a made-up ROI number in front of a Cognizant panel is the single fastest way to lose the room.

SAY THIS

We do not have an ROI multiple and we are not going to invent one. What we can tell you is what the failure costs, because it is documented: TSB's failed migration ran to roughly £400 million all-in — a £48.65 million fine, £32.7 million in customer redress, the CEO resigned, and the regulator's finding was about risk management and assurance, not about programming. That is the exact gap this fills, and our cost is two orders of magnitude below it. In India the currency is not just money: after its 2020 outages HDFC Bank lost its entire digital product roadmap for over a year by regulatory order.

What the buyer actually gets

1. **A decision they currently cannot make.** The buyer is the person who signs the cutover memo and today has nothing to attach to it. We produce the artefact that unblocks a stalled approval. That is the purchase. **ASSUMPTION** on the pain, sourced on the role
2. **A large cut in review cost, by mechanism.** Every extracted rule carries the line numbers it came from, so a reviewer confirms or rejects it by reading three lines instead of four thousand. Comparable automation of one pipeline stage — AWS's own ADP case — cut rule-extraction time by 80% and manual effort by over 90%. Attribute that to AWS; it is their case study, not our measurement. **SOURCED**
3. **Two assets retained even if the migration is cancelled.** The specification and the regression suite. The bank ends up with documentation for a system it previously could not touch. This is what de-risks the purchase: the spend is not contingent on the migration completing.
4. **Compliance findings nobody else can produce.** Because we check the legacy against the regulator, not just against the rewrite. A breach that has run for forty years is invisible to every two-way tool on the market. **SOURCED**

If someone insists on a rupee figure

There is exactly one honest construction, and the dossier already specifies it: engineer-months multiplied by an assumed loaded monthly rate, *with the rate stated on the slide as an assumption*. Our own build is four engineers for twelve weeks; a bank pilot is two engineers plus SME time for about six months. If we want a number for the finals, we should agree the rate as a team and label it — not derive one on stage.

The framing that wins this question: we are not asking the bank to find a new budget. The modernisation budget is already approved — that is the ~\$1bn BCG figure. What stalls it is the assurance step between “the new code exists” and “we are willing to run it on real money,” and that step has no product category today; it is absorbed into consulting hours and schedule risk. We are unblocking committed spend, not competing for new spend.

What still needs doing before the finals

Carried over from the dossier's own open-items list, plus what this meeting added:

- **Agree a loaded monthly engineer rate** if we want any rupee figure at all, and put the assumption on the slide. Otherwise commit to answering Q5 with the downside framing only. *(New from this meeting.)*
- **Decide whether the recurring-revenue answer goes on a slide or stays as a Q&A answer.** Recommendation: keep it in Q&A. Putting an unpriced business model on a slide invites the numbers question we cannot answer; answering it well when asked reads as considered. *(New from this meeting.)*
- **Pin the 5%-vs-7-9% IT-spend figure** to a named report and year, or swap in the RBI ombudsman figure — 40,000+ mobile and internet banking complaints across FY22 and FY23, which is regulator-sourced and currently unused on the deck. This is the weakest citation we are showing.
- **Validate the four stakeholder pain statements with one real practitioner** — a core-banking engineer or a bank IT auditor. One conversation upgrades all four rows from our inference to testimony.